Experiment 9: Bidirectional Search

CSA1717 – Artificial Intelligence

Name: Hemalakshmi H

Reg.No.: 192571049

Aim:

To implement Bidirectional Search, a graph search technique that simultaneously explores from the start node and the goal node, reducing the search space and improving efficiency in finding the shortest path.

Algorithm:

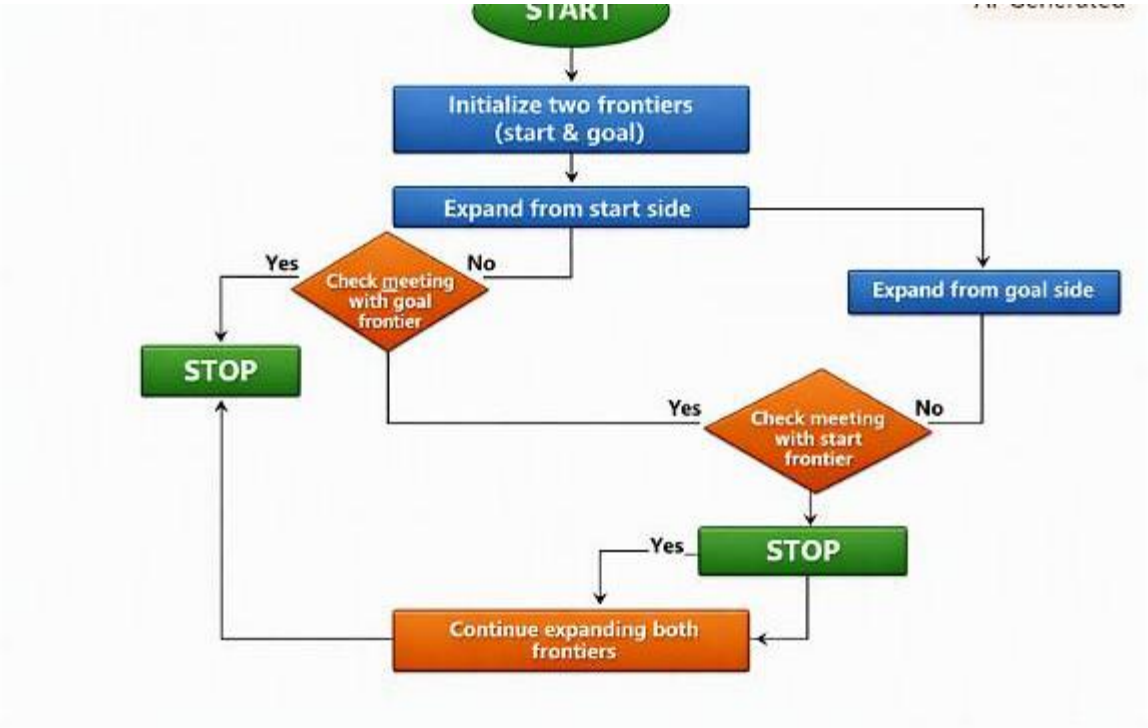
Bidirectional Search Algorithm

1. Initialize two frontiers:
    - front_start → begins from the start node
    - front_goal → begins from the goal node
2. Maintain two visited sets:
    - visited_start
    - visited_goal
3. Expand nodes alternately from both frontiers:
    - Remove a path from front_start
    - Expand its last node
    - Add new paths to the frontier
    - Check if any expanded node appears in visited_goal
        - If yes → meeting point found
4. Repeat the same expansion for front_goal:
    - Remove a path
    - Expand
    - Add new paths
    - Check if any expanded node appears in visited_start
5. When a meeting point is found:

- Combine the forward path and the reversed backward path
- Return the complete path

6. If both frontiers become empty → no path exists.

Flowchat:



Source code:

```
from collections import deque


def bidirectional_search(graph, start, goal):
    if start == goal:
        return [start]


    # Frontiers for forward and backward search
    front_start = deque([[start]])
    front_goal = deque([[goal]])
```

```
# Visited sets
visited_start = {start}
visited_goal = {goal}

while front_start and front_goal:

    # ---- Expand from start side ----
    path_start = front_start.popleft()
    node_start = path_start[-1]

    for neighbor in graph[node_start]:
        if neighbor not in visited_start:
            visited_start.add(neighbor)
            new_path = path_start + [neighbor]
            front_start.append(new_path)

            # Meeting point found
            if neighbor in visited_goal:
                # Find the matching path from goal side
                for p in front_goal:
                    if p[-1] == neighbor:
                        return new_path + p[-2::-1]

    # ---- Expand from goal side ----
    path_goal = front_goal.popleft()
    node_goal = path_goal[-1]

    for neighbor in graph[node_goal]:
        if neighbor not in visited_goal:
```

```
                visited_goal.add(neighbor)
                new_path = path_goal + [neighbor]
                front_goal.append(new_path)

                # Meeting point found
                if neighbor in visited_start:
                    # Find the matching path from start side
                    for p in front_start:
                        if p[-1] == neighbor:
                            return p + new_path[-2::-1]

    return None
# ----------------------------
# Example bidirectional graph
# ----------------------------
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'G'],
    'F': ['C'],
    'G': ['E']
}

path = bidirectional_search(graph, 'A', 'G')
print("Path found:", path)
```

Output:

```
Enter "help" below or click "help" above for more information.
>>>
= RESTART: C:/Users/hemal/OneDrive/Documents/SIMATS/AI/AI_PY/Bidirectional_AI.py
Path found: ['A', 'B', 'E', 'G']
>>>
```

Result:

The Bidirectional Search successfully finds the shortest path between **A** and **G** by simultaneously exploring from both ends. The two searches meet at node **E**, producing the final path:

A → B → E → G

This confirms that Bidirectional Search reduces search time by exploring from both directions and meeting in the middle.